

SIMATS ENGINEERING

ASSESSMENT TOOL 3

COURSE NAME: OBJECT ORIENTED ANALYSIS AND DESIGN
COURSE CODE: CSA1102

COURSE OUTCOME COVERED

CO4:

implement object-oriented system layers using design principles and patterns, and integrate access and view layers with OODBMS (*BL5*)

Assessment Tool Weightage: 30%

QUESTIONS: 6

Total Marks:30

Annexure A

Reverse Engineering Task

Code of Conduct:

I E. Sakthi priya (192520041) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

E. Sakthi priya

Annexure A

Reverse Engineering Task

1. Legacy E-Learning Management System

An existing **E-Learning Management System** has large modules responsible for student registration, course management, content delivery, assignment evaluation, and result generation. User interface code and business rules are tightly coupled, making modifications difficult.

Question:

Reverse engineer the existing system by identifying appropriate classes, responsibilities, attributes, methods, and relationships. Redesign the system using **object-oriented principles and layered architecture**, and explain how the redesigned structure improves cohesion, coupling, maintainability, and scalability.

2. Legacy Customer Support System

A customer support application uses multiple conditional statements to handle support requests through email, chatbot, telephone, and social media channels. Adding a new support channel requires changes to several existing modules.

Question:

Analyze the existing implementation and identify its object-oriented design limitations. Reverse engineer and refactor the system using suitable **behavioral design patterns**, and explain how the redesigned solution improves flexibility, extensibility, loose coupling, and maintainability.

3. Legacy Banking Transaction System

A banking application contains account management, transaction validation, fund transfer, loan processing, and transaction reporting within a few tightly coupled classes. Database operations are also embedded directly within business methods.

Question:

Reverse engineer the system by identifying appropriate **domain classes, service classes, and data access components**. Redesign the application using suitable layers and design patterns, and explain how the redesigned architecture improves separation of concerns, testability, and system maintainability.

4. Legacy Hotel Reservation System

A hotel reservation application directly performs OODBMS operations from reservation screens. The same database queries for rooms, guests, bookings, and payments are repeated across multiple modules.

Question:

Analyze the existing architecture and identify problems related to **coupling, duplication, and persistence management**. Reverse engineer the system by introducing suitable **DAO, Repository, or Data Access patterns**, and explain how the redesigned solution improves modularity, reusability, and OODBMS integration.

5. Legacy Transportation Management System

A transportation application contains separate modules for buses, trains, taxis, and rental vehicles. Object creation logic for each transportation type is duplicated across several parts of the application, making it difficult to introduce new transportation services.

Question:

Reverse engineer the system by identifying common abstractions, classes, and object relationships. Apply suitable **creational design patterns**, such as Factory Method or Abstract Factory, to redesign object creation and explain how the new design improves extensibility, scalability, and code reuse.

6. Legacy Healthcare Monitoring System

A healthcare monitoring application collects patient information, processes sensor readings, generates alerts, stores medical records, and produces reports. Several classes perform multiple unrelated tasks and directly depend on concrete implementations of monitoring devices and storage components.

Question:

Reverse engineer the application by identifying violations of **SOLID principles**, particularly SRP, OCP, and DIP. Redesign the classes using suitable abstractions, interfaces, and design patterns, and explain how the redesigned system improves cohesion, loose coupling, flexibility, testability, and maintainability.

RUBRICS FOR CALCULATION

Reverse Engineering Task

Criteria	Weightage	Exemplary (4)	Proficient (3)	Developing (2)	Beginning (1)
Understanding of System	20%	Demonstrates clear and in-depth understanding of the system/product and its functionality	Shows good understanding with minor gaps	Partial understanding; misses key aspects	Lacks understanding of the system/product
Decomposition	25%	Effectively breaks down the system into components with detailed analysis	Decomposes system with minor gaps in analysis	Limited or incomplete decomposition	Unable to analyze or break down system
Identification of Components	20%	Accurately identifies all components and explains their functions clearly	Identifies most components with minor errors	Identifies few components; explanations are weak	Unable to identify components or functions
Reconstruction	20%	Successfully reconstructs or models the system with accuracy and logic	Reconstruction is mostly correct with minor issues	Partial reconstruction; lacks clarity	Unable to reconstruct or model system
Documentation	15%	Well-organized, clear documentation with diagrams and structured explanation	Documentation is adequate with minor issues	Poorly organized or incomplete documentation	No proper documentation or presentation

1. Legacy E-Learning Management System

The existing E-Learning Management System contains modules for student registration, course management, content delivery, assignment evaluation and result generation. In the legacy system, UI code and business logic are tightly coupled, making modification and maintenance difficult.

Reverse Engineering

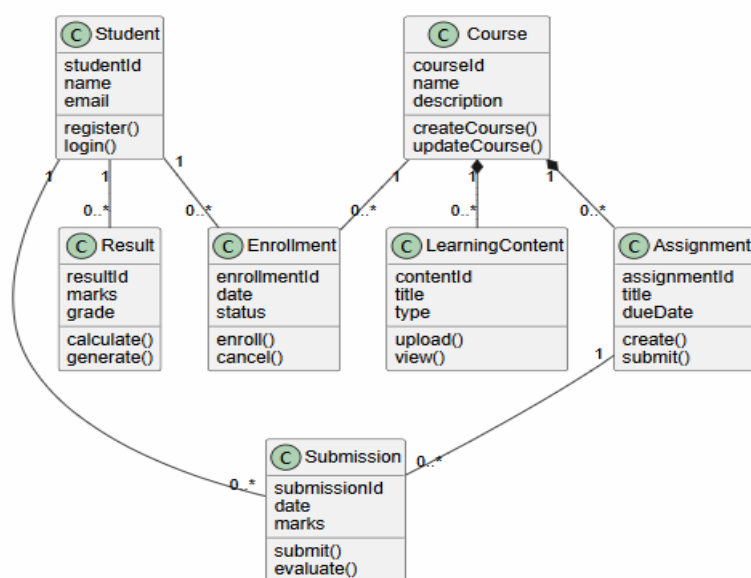
The main classes identified are:

Class	Attributes	Main Methods
Student	studentId, name, email	register(), login(), viewCourses()
Course	courseId, name, description	createCourse(), updateCourse()
Enrollment	enrollmentId, date, status	enroll(), cancel()
LearningContent	contentId, title, type	upload(), view()
Assignment	assignmentId, title, dueDate	create(), submit()
Submission	submissionId, date, marks	submit(), evaluate()
Result	resultId, marks, grade	calculate(), generate()

Relationships

- A Student can have many Enrollments.
- A Course can have many Enrollments.
- A Course contains LearningContent and Assignments.
- An Assignment can have many Submissions.
- A Student can have multiple Results.

UML Class Diagram



Layered Architecture

The redesigned system is divided into four layers:

1. Presentation Layer – handles user interface.
2. Business Layer – contains application rules and services.
3. Data Access Layer – communicates with the database.
4. Database Layer – stores students, courses, assignments and results.

Flow:

User → Presentation → Business → Data Access → Database

Benefits

- High cohesion: Each class performs a specific responsibility.
- Low coupling: Layers communicate through defined interfaces.
- Maintainability: Changes can be made to one module without affecting the entire system.
- Scalability: New features such as online exams and certificates can be added easily.
- Reusability: Business services can be reused by different interfaces.

Thus, the redesigned object-oriented and layered system is easier to maintain, test, modify and scale than the tightly coupled legacy system.

2. Legacy Customer Support System

The existing Customer Support System uses multiple conditional statements such as if-else or switch to process requests from email, chatbot, telephone and social media. Adding a new channel requires modifying existing modules, making the system difficult to maintain.

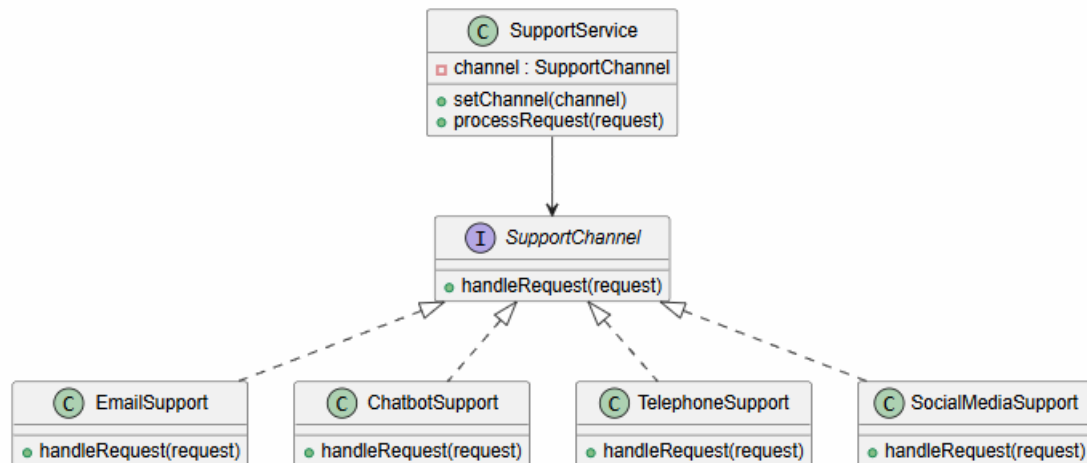
Object-Oriented Design Limitations

- Excessive if-else/switch statements.
- High coupling between support channels and the main processing logic.
- Low flexibility when adding new channels.
- Existing code must be modified whenever a new channel is introduced.
- Difficult to test and maintain individual channels.

Suitable Behavioral Design Pattern

- The Strategy Pattern is suitable because each support channel can be treated as a different strategy for processing a request.
- Create a common interface called SupportChannel. Email, Chatbot, Telephone and SocialMedia implement this interface.
- The SupportService uses the selected strategy without knowing the internal details of each channel.

UML Class Diagram



Refactored Design

Instead of checking the channel using conditional statements, **SupportService** delegates the request to the selected **SupportChannel** object.

For example:

- **SupportService** → **SupportChannel** → **EmailSupport** (or)
- **SupportService** → **SupportChannel** → **ChatbotSupport**

A new channel such as WhatsApp can be added by creating a new class implementing **SupportChannel**, without modifying the existing service.

Benefits

- **Flexibility:** Different support strategies can be selected dynamically.
- **Extensibility:** New channels can be added without changing existing classes.
- **Loose Coupling:** **SupportService** depends only on the **SupportChannel** interface.
- **Maintainability:** Each channel has its own implementation and responsibility.
- **Better Testing:** Individual support channels can be tested independently.

Thus, the Strategy Pattern removes complex conditional logic and provides a flexible, loosely coupled and maintainable customer support system.

3. Legacy Banking Transaction System

The existing banking system combines account management, transaction validation, fund transfer, loan processing, reporting and database operations in a few tightly coupled classes. This creates poor separation of concerns and makes testing and maintenance difficult.

Reverse Engineering

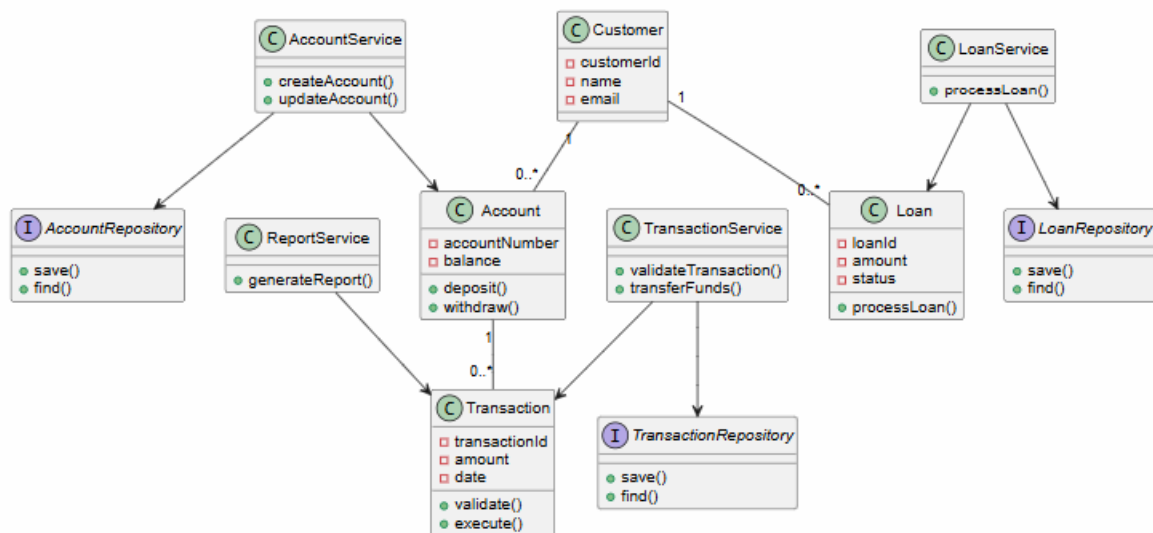
The system can be divided into the following components:

Type	Classes	Responsibility
Domain Classes	Account, Transaction, Loan, Customer	Store banking data and core behavior
Service Classes	AccountService, TransactionService, LoanService, ReportService	Implement business rules
Data Access	AccountRepository, TransactionRepository, LoanRepository	Handle database operations

Suitable Design Patterns

- Repository Pattern: Separates database operations from business logic.
- Service Layer Pattern: Keeps business rules inside service classes instead of domain-independent UI/database code.
- Factory Pattern: Can be used to create different transaction types such as Deposit, Withdrawal and Transfer.

UML Class Diagram



Layered Architecture

The redesigned system uses:

1. Presentation Layer – handles user requests and displays results.
2. Service Layer – contains transaction, account and loan business logic.
3. Domain Layer – contains Account, Transaction, Loan and Customer classes.
4. Data Access Layer – repositories communicate with the database.
5. Database Layer – stores banking information.

Flow:

User → Presentation → Service → Domain/Repository → Database

Benefits

- Separation of concerns: UI, business logic and database operations are separated.
- Testability: Services can be tested without directly accessing the database.
- Maintainability: Database changes do not require changes to business logic.
- Low coupling: Services depend on repository interfaces instead of database implementation.
- Scalability: New transaction types and banking services can be added easily.

Thus, the layered architecture with the Repository and Service Layer patterns produces a more modular, testable and maintainable banking system.

4. Legacy Hotel Reservation System

The existing hotel reservation system directly performs OODBMS operations from reservation screens. Database queries for rooms, guests, bookings and payments are repeated in multiple modules.

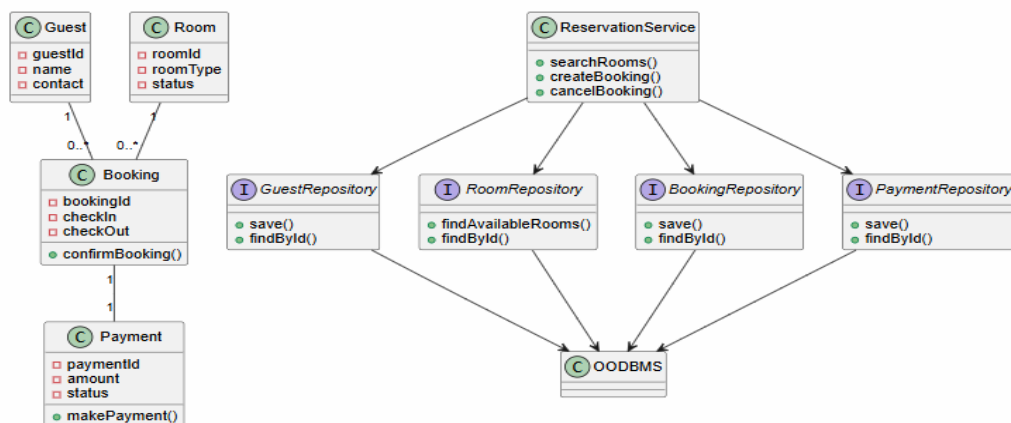
Problems in Existing System

- High coupling: UI is directly connected to the database.
- Code duplication: Same database queries are repeated.
- Poor persistence management: Database operations are mixed with application logic.
- Low reusability: Database code cannot be easily reused.
- Difficult maintenance: Changes in the OODBMS require changes in several modules.

Suitable Design Pattern

The Repository Pattern is suitable for separating persistence operations from business logic. A repository provides common operations for each domain object, while the UI communicates with service classes instead of directly accessing the OODBMS.

UML Class Diagram



Refactored Architecture

The redesigned system follows:

Reservation UI → Reservation Service → Repository/DAO → OODBMS

- UI Layer: Handles reservation screens.
- Service Layer: Contains booking and reservation logic.
- Repository Layer: Performs database operations.
- OODBMS Layer: Stores guests, rooms, bookings and payments.

Benefits

- Modularity: UI and database code are separated.
- Reusability: Common database operations are implemented once.
- Reduced duplication: Repositories can be reused by multiple services.
- Easy maintenance: OODBMS changes are isolated to the data access layer.
- Better integration: Repository classes provide a consistent interface to the OODBMS.

Thus, the Repository/DAO pattern reduces coupling and duplication while providing better persistence management, modularity and maintainability.

5. Legacy Transportation Management System

The existing system has separate modules for buses, trains, taxis and rental vehicles. Object creation code is repeated in different modules, resulting in code duplication and difficulty when adding new transportation types.

Common Abstraction

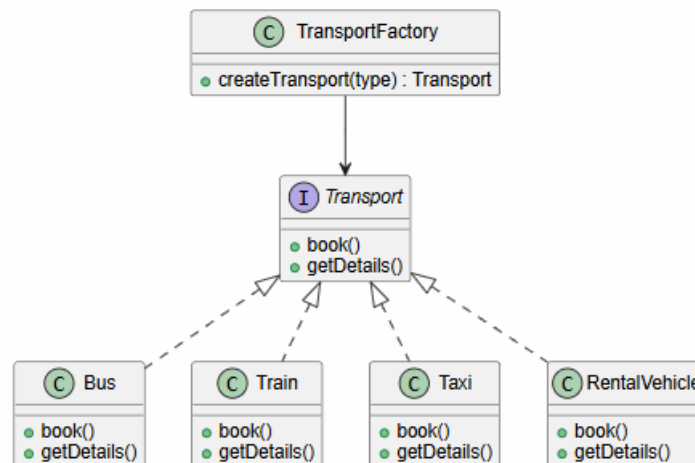
A common Transport interface can be created for all transportation types.

Class	Main Responsibility
Transport	Common transport operations
Bus	Bus-specific operations
Train	Train-specific operations
Taxi	Taxi-specific operations
RentalVehicle	Rental vehicle operations
TransportFactory	Creates required transport objects

Suitable Design Pattern

The Factory Method Pattern is suitable because object creation is centralized in a factory. The client does not need to know how individual transportation objects are created.

UML Class Diagram



Refactored Design

- Instead of creating objects directly:
- `new Bus()`, `new Train()`, `new Taxi()`
- the application requests an object through:
- `TransportFactory.createTransport(type)`
- The factory creates the required transportation object and returns it as a `Transport`.

Benefits

- **Extensibility:** New transportation types can be added easily.
- **Code Reuse:** Common operations are defined in the `Transport` interface.
- **Reduced Duplication:** Object creation logic is centralized.
- **Low Coupling:** Client code does not depend directly on concrete classes.
- **Scalability:** Additional services such as flights or metro can be integrated with minimal changes.

Thus, the Factory Method pattern provides a flexible and reusable approach for creating transportation objects and makes the system easier to extend and maintain.

6. Legacy Healthcare Monitoring System

The existing healthcare monitoring system contains classes that perform multiple unrelated tasks such as collecting sensor data, processing readings, generating alerts, storing records and creating reports. Classes also directly depend on specific monitoring devices and storage implementations.

SOLID Violations

- SRP: One class performs multiple responsibilities such as monitoring, storage and reporting.
- OCP: Adding a new sensor or alert type requires modifying existing classes.
- DIP: High-level classes directly depend on concrete sensors and storage components.

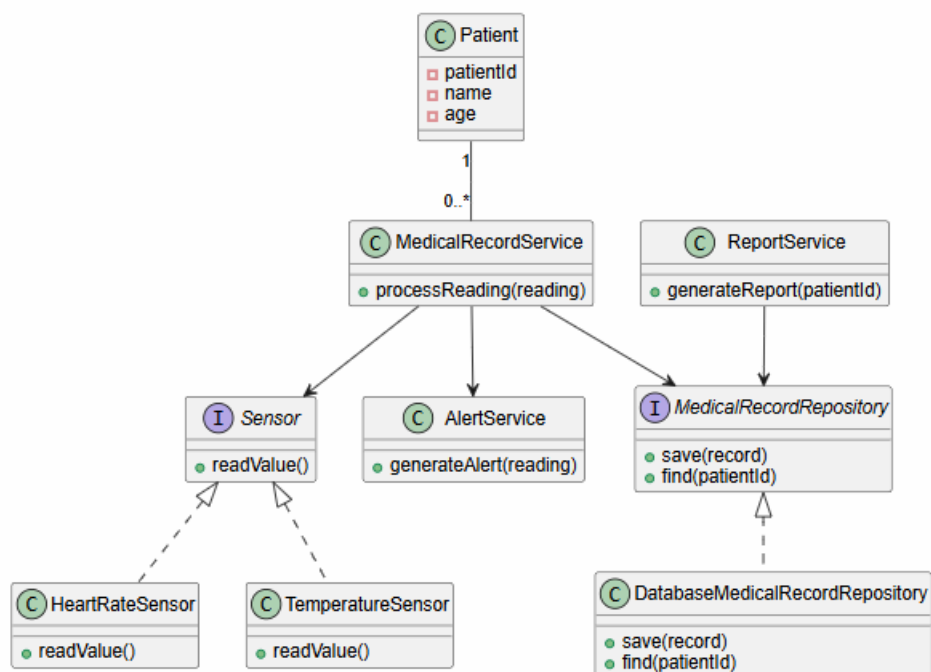
Refactored Classes

Class/Interface	Responsibility
Patient	Stores patient information
Sensor	Common interface for sensors
HeartRateSensor	Collects heart-rate readings
TemperatureSensor	Collects temperature readings
AlertService	Generates alerts
MedicalRecordService	Processes medical records
ReportService	Generates reports
MedicalRecordRepository	Stores and retrieves records

Suitable Design

Use interfaces and Dependency Injection to follow SOLID principles. The system depends on abstractions such as Sensor and Medical Record Repository rather than concrete implementations.

UML Class Diagram



Benefits

- High cohesion: Each class has a focused responsibility.
- Loose coupling: Classes depend on interfaces instead of concrete implementations.
- Flexibility: New sensors and storage systems can be added easily.
- Testability: Mock sensors and repositories can be used for testing.
- Maintainability: Changes to one component have minimal effect on others.

Thus, applying SRP, OCP and DIP with interfaces and dependency injection produces a more flexible, testable and maintainable healthcare monitoring system.